Curiculum sponsored by INTEL

Advanced ASIC Design

Laboratory Instruction

Exercise no 6:
SPYGLASS CDC & RDC

1. SG Lint
2. VCS
3. FC Synthesis and Implementation – Part1
4. FC Synthesis and Implementation – Part2
5. FC Synthesis and Implementation – Part3
6. **SG CDC & RDC**

I. Two-stage synchronizer/FF synchronizer

1. An example of the synchronizer

Download the "ff_synch.zip" file from the remote education platform. It contains a model of the synchronizer along with a testbench that allows you to observe its operation and a script that runs the simulation ("my_synch_sym.sh"). The testbench allows you to simply change the frequency of the source (clk_a) and destination (clk_b) clocks by editing the **CLK_a_F_HZ**, **CLK_b_F_HZ** parameters. Take a look at the file "my_synchornizer.v".
Run simulations at the default settings (moving from a 50MHz clock domain to 80MHz) and observe the circuit's behavior. **In the report (Point 1), answer the question: in the simulation, is the data from clock domain A, correctly transferred to clock domain B? Embed time waveforms to confirm this answer.**

2. SpyGlass as a CDC verification tool

The SpyGlass tool has rules to verify the designed CDC synchronizers. After adding design files to this tool and setting the master module (in this case, "my_synchornizer"), in the "Goal setup" tab you can select rules to support the design of CDC synchronizers.

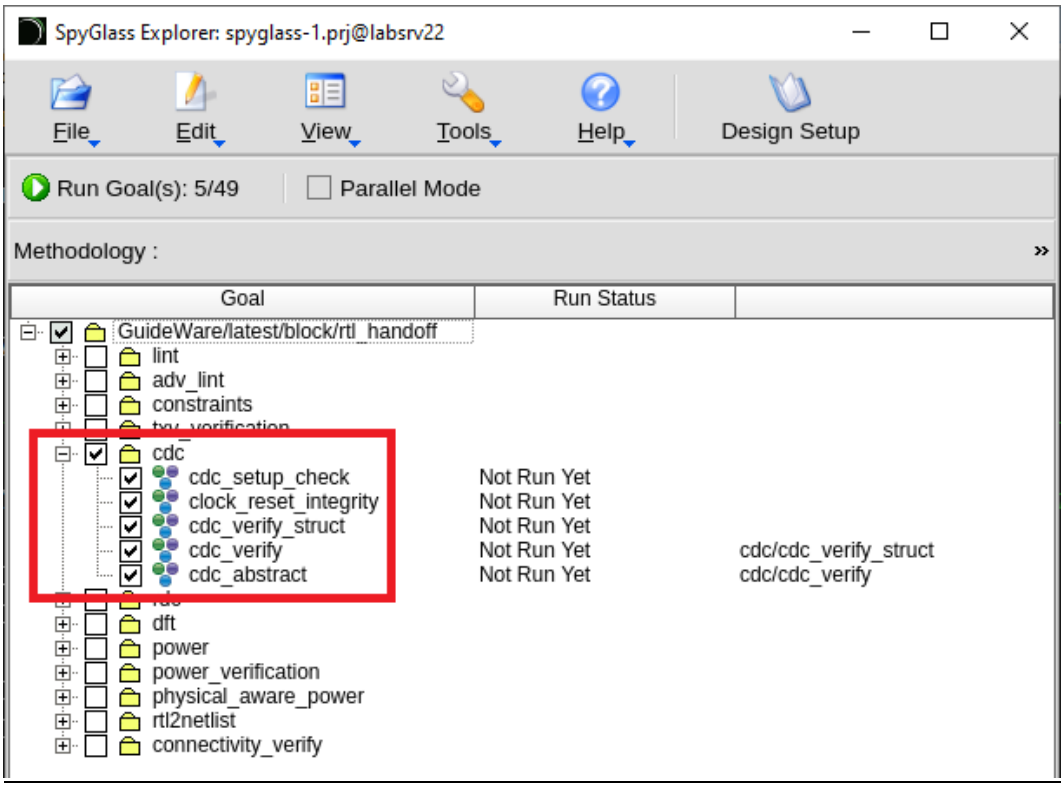

Fig. 1 CDC rules.

In order to correctly interpret CDC problems, information about the input signals of the module being checked must be supplied to the SpyGlass tool. This data is saved in a file "*.sgdc" which is then attached together with the source files.
The minimum content of this file is information on clock signals and reset signals. Information about additional settings possible to include in the "*.sgdc" file can be found in the SpyGlass help (Help button). In the example under consideration, this file will look as follows:

```
1  current_design my_synchronizer
2  clock -name my_synchronizer.i_clk_a -period 20 -edge {0 10}
3  clock -name my_synchronizer.i_clk_b -period 12.5 -edge {0 6.25}
4  reset -name my_synchronizer.i_rst_a -value 0
5  reset -name my_synchronizer.i_rst_b -value 0
```

*Fig. 2 The file *.sgdc.*

current_design "module_name" – This is the name of the investigated module

clock – clock_parameters

-name "path" – lokalizacja portu zegara

-period "ns" – the period of the clock signal expressed in ns

-edge "{ns ns}" – duty factor, the first value is the time in occurrence in nanoseconds of the rising edge, and the second is the time of occurrence of the falling edge. With a period of 10ns, {0 5} means a waveform with a duty factor of 50%

reset – the parameters of the reset signal

-name "path" – location of the reset port

- value – determines the active level of the reset signal (0 or 1)

Add file *"*.sgdc"* to your project ad run the CDC rules checking procedure.

3. Results analysis

After checking the rules in the "cdc_verify_struct" category, the SpyGlass tool will search for the violation: "Ac_unsync01". Please remember to enable the help window (if it is not enabled) "View -> Windows -> Help Viewer", it will allow you to view help for the current (highlighted) bug.

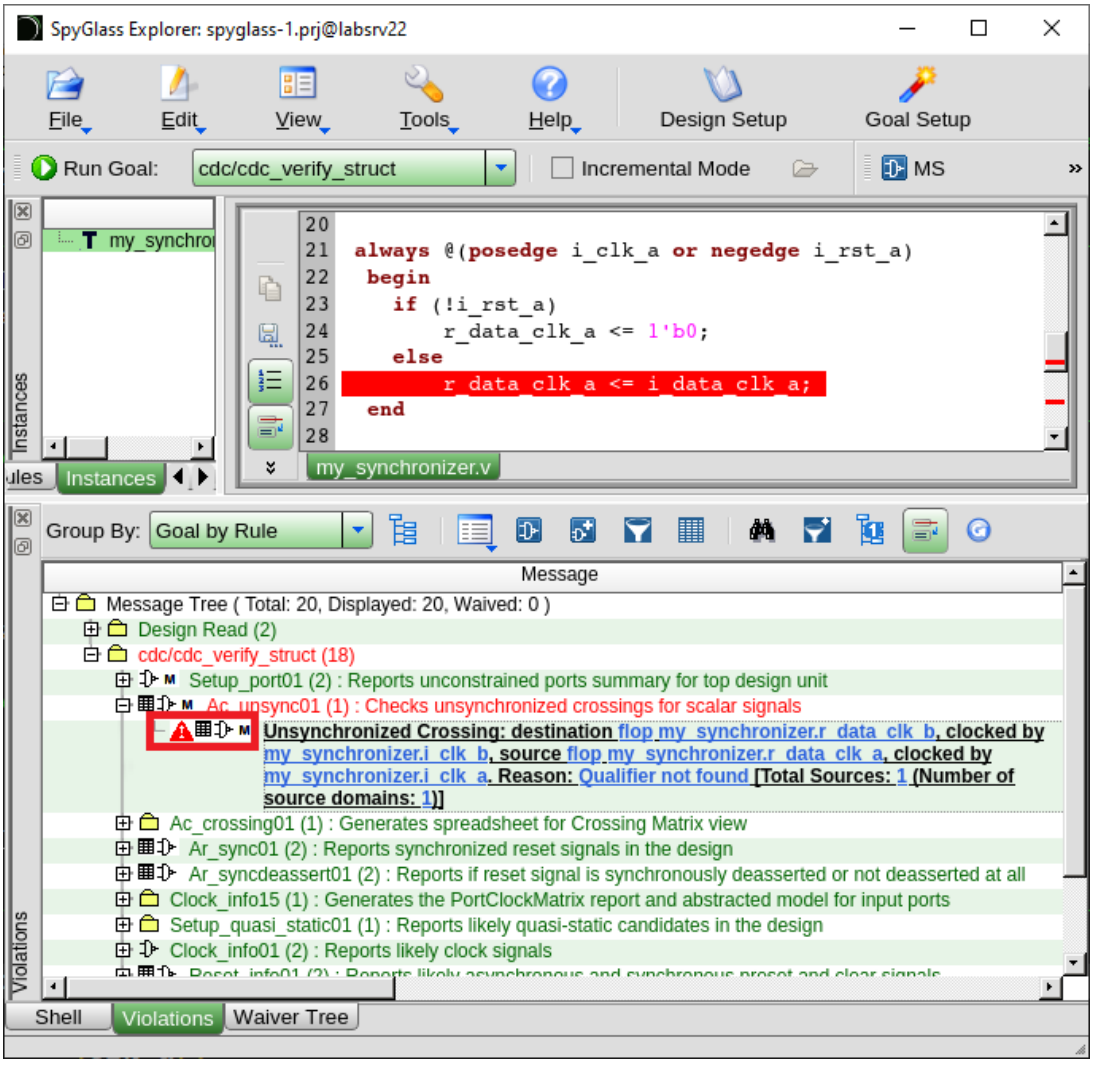

Fig. 3 Ac_unsync01 rule violation.

When you expand the violation and click on the selected area, you can for example open a diagram (when you select "*Incremental Schematic*") showing where the synchronization is missing:

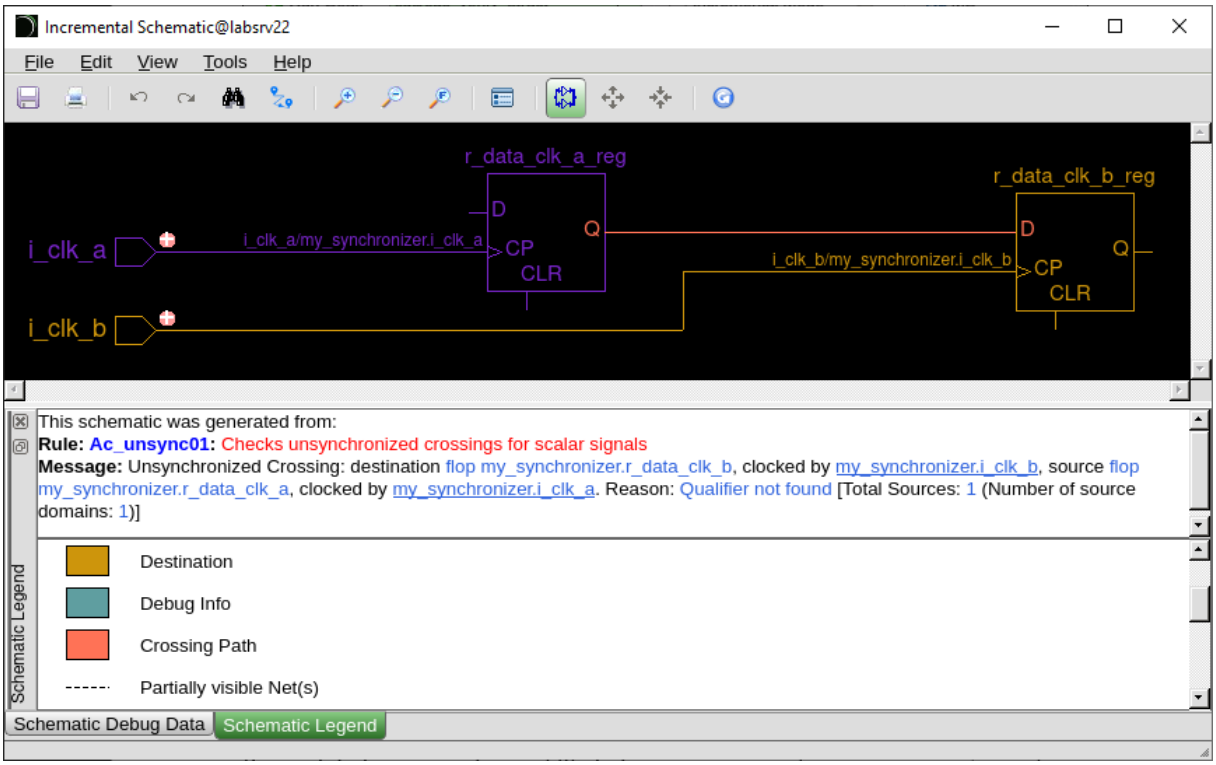

Fig. 4 The scheme showing the place where the synchronization is missing.

In the report (Point 2), describe and explain why this error occurs even though on the simulation the circuit behaves correctly. Next, modify the layout of the "my_synchornizer" module so that it is correctly synchronized. Then, put the corrected code in the report. Also test the circuit on the simulation and document it by adding time waveforms to the report.

4. Signal passing from a faster clock domain to the slower clock domain

Modify the testbench file "my_synchornizer_tb.v" so that there is a transition from a faster clock domain to a slower one (e.g., from 80MHz to 50MHz). Run the simulation. Run the SpyGlass tool with the changed "*.sgdc" file.

In the report (Point 3), answer the questions, documenting the answers with screen shots: Was the data correctly transferred from clock domain A to clock domain B during the simulation? Did the Spyglass tool report any violations? What is the nature of the problem?

Project:

Please spend a maximum of 2h on the design. If this time is not enough, please move on, and in the report describe only what you managed to do.

Modify the synchronizer ("my_synchronizer") so that it allows error-free transmission of one bit from a faster clock domain to a slower one. The testbench contains the "next_data_clk_a" signal. It allows you to delay sending new data from clock domain A. The new data will be updated ("data_a") only if the signal "next_data_clk_a", will have a value of one during the rising edge of clock A. Thus, it is necessary to drive this signal so that there is no loss of data resulting from the different clock frequency of clocks A and B.

In the report (Point 4), place the code of the modified synchronizer and the time waveforms showing the correct operation of the circuit. Include the contents of the created *.sgdc file. Then present a screen shot from the Spyglass tool showing the lack of violations found.

Include a short description of the designed layout. Describe the conclusions and thoughts coming from the layout design process.

5. Synchronization of two reset signals in one clock domain.

Download the file "ff_synch_RDC.zip" from the remote education platform. The contents and file names are very similar to the first example mentioned above. However, this example involves only one clock domain, and the issue concerns the synchronization of flip-flop reset signals. The schematic of the initial solution is shown in Fig. 5.

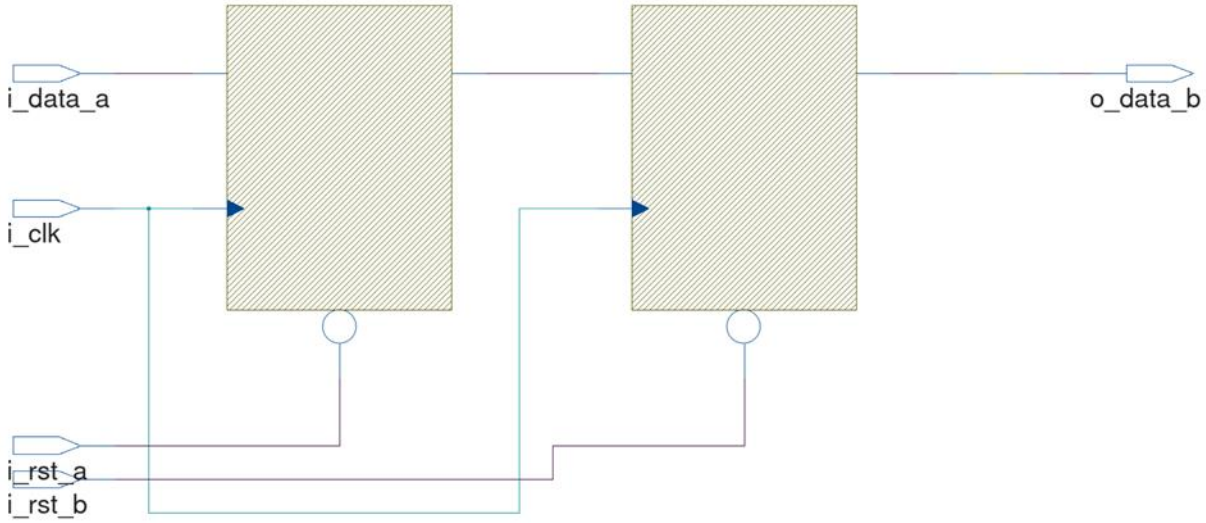

Fig. 5 Reset synchronizator

The downloaded example should be verified using the **Spyglass** tool by running all **CDC** and **RDC** rules. Any violations that occur should be resolved by adding a **two-stage synchronizer** to the project.

Fig. 6 CDC & RDC rules

In the report (Point 5), explain why synchronization of flip-flop resets is required, even though the resets in the discussed examples are asynchronous. The report should include screenshots from the Spyglass tool showing the detected RDC violations. Then, include the code of the designed reset signal synchronizer, followed by a screenshot confirming the elimination of the previously reported violations after synchronization was implemented.

6. Two clock domains, one global asynchronous reset.

Download the file "ff_synch_RDC2.zip" from the remote education platform. The included example presents a case of synchronizing a signal originating from clock domain A to clock domain B. The entire system should be reset using a single global asynchronous reset signal.

You need to add a data synchronizer to the design, as well as circuits generating local reset signals for both clock domain A and clock domain B. The schematic of the original solution is shown in Fig. 7.

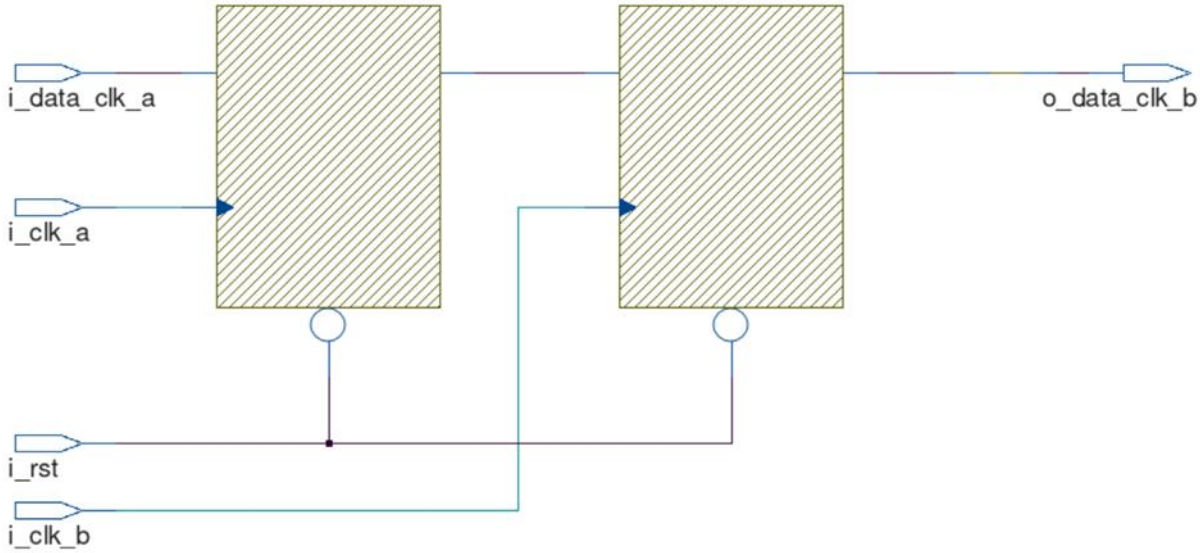

Fig. 7 Global reset

In the report (Point 6), list the errors generated by the CDC and RDC rules in the SpyGlass software, along with a brief description of how each error was resolved.

II. Valid/Ready Protocol

In the next part of the laboratory, mechanisms for transmitting multi-bit data from one clock domain to another will be discussed. In order to standardize the way data is received and transmitted, the Valid/Ready protocol will be used in these examples. The synchronizer being studied has a V/R input and output interface, and the test environment has a virtual V/R transmitter and receiver.

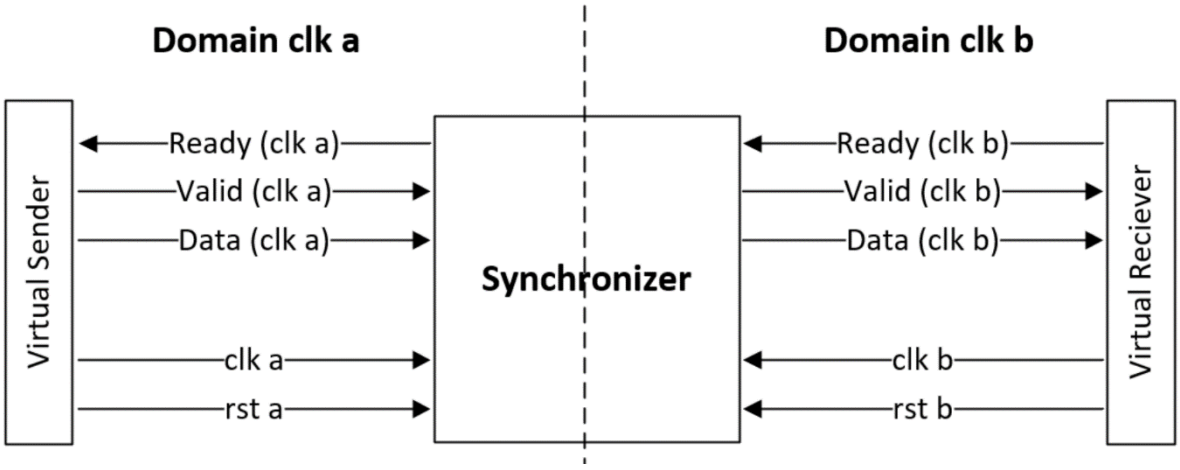

Fig. 8 The scheme of the synchronizer based on the Valid/Ready protocol.

This protocol works on a very simple principle: If the receiver is ready to receive data it sets the "Ready" signal to 1. If the transmitter is ready to send data it sets the "Valid" signal and the data being sent to the "Data" line. Transmission occurs only when the Valid and Ready signals are in a high state.

1. Valid/Ready testbench

To demonstrate the operation of the synchronizers, a test environment containing a virtual V/R transmitter and receiver was designed (Fig. 5). The behavior of the virtual receivers and transmitters can be changed through the parameters described at the beginning of the testbench. Using the parameters: MIN_DELAY_VALID_A and MAX_DELAY_VALID_A, you can set the transmission delay

of the virtual transmitter. This is the time for setting the "Valid" flag and issuing new data (in clock cycles) after a valid data transmission. This time is randomly generated from the range of these parameters (MAX and MIN). If these values are the same, this parameter is fixed (it is not randomized). For example: if both of these values are set to "0" after each "Ready" return signal (in the next clock cycle) the "Valid" flag will be set along with the new data. Thus, the transmitter will not have any delay. In the same way, the delay of the virtual receiver can be adjusted via the MIN_DELAY_READY_B and MAX_DELAY_READY_B parameters.

During the simulation, the console prints out information about what data is sent and received and when. This will allow a simple comparison of the speed and behavior of the synchronizers being checked.

III. REQ/ACK Synchronizer

Download the "req_ack.zip" file from the remote education platform. It contains a model of a multi-bit req/ack synchronizer, along with a testbench that allows you to observe its operation, and a script that runs the simulation ("req_ack_sym.sh").

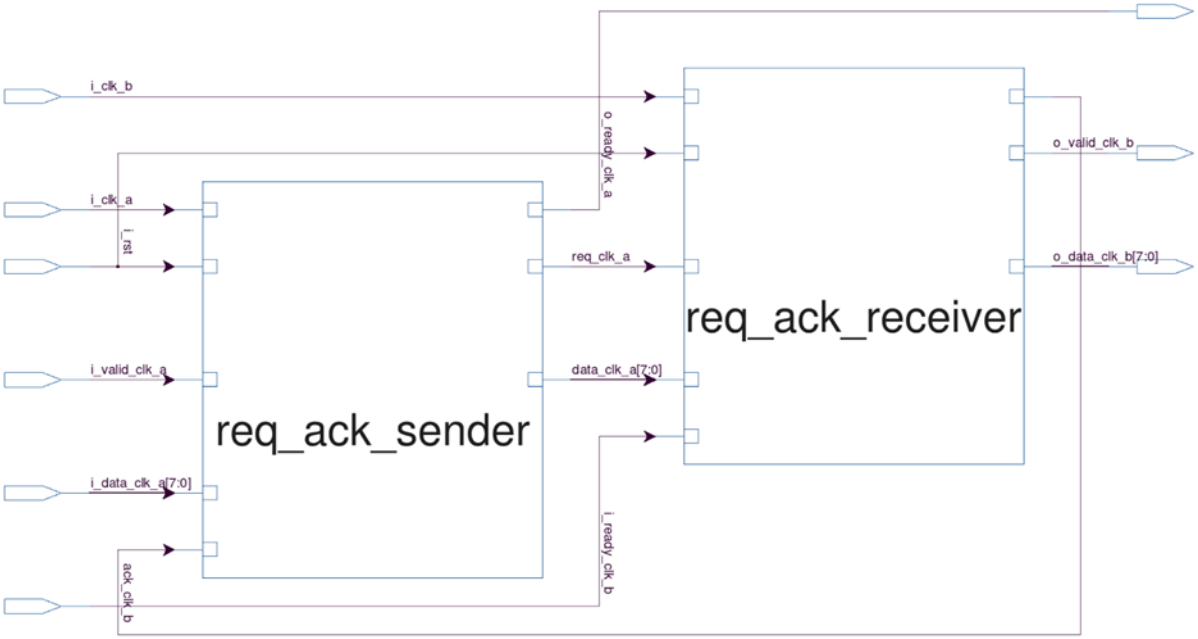

Fig. 9 Initial schematic of the REQ/ACK solution.

Run the simulation and become familiar with how the synchronizer works. Use the SpyGlass tool to search for CDC violations. Correct the detected errors.

Analyze the operation of the corrected synchronizer in the simulation under different conditions (for various parameters of clock frequency and V/R protocol).

In the report (Point 7), describe the operation of the synchronizer. What was the error detected by the Spyglass tool? How to fix it? Why are two-step synchronizers required only on ack and req lines? In the report, document the correct operation of the synchronizer through: simulation screens and SpyGlass software screens.

IV. FIFO Synchronizer

Download the "fifo.zip" file from the remote education platform. It contains a model of a multi-bit fifo synchronizer along with a testbench that allows you to observe its operation, and a script that runs the simulation ("fifo_sym.sh").

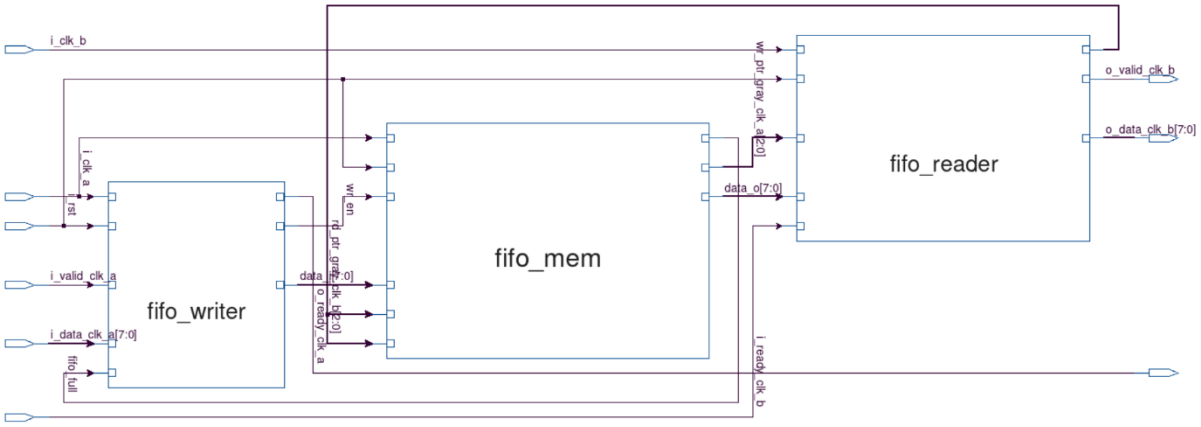

Fig. 10 Initial schematic of the FIFO solution

Run the simulation and become familiar with the operation of the synchronizer. Use the SpyGlass tool to search for CDC violations. Correct the detected errors.

Familiarize yourself with the performance of the corrected synchronizer in the simulation under different conditions (varying the parameters of clock signal frequency and V/R protocol). Please also check the effect of FIFO memory depth on the synchronizer operation.

In the report (Point 8), describe the operation of the synchronizer. What was the error detected by the Spyglass tool? How to fix it? Why are FIFO pointers written in Grey's code? How to select the depth of the FIFO according to the load of the synchronizer? In the report, document the correct operation of the synchronizer by: simulation screenshots and SpyGlass software screenshots.

V. Comparison of REQ/ACK and FIFO Synchronizers

Please propose some test scenarios (testbench parameters) and run both synchronizers in them.

In the report (Point 9), list the proposed scenarios (parameter sets). Present the total data transmission time for each scenario in the form of a table. Indicate which synchronizer is faster. Compare the advantages and disadvantages of both synchronizers. Finally, discuss possible improvements that could be made to the provided synchronizer implementations to make them operate faster.